1. Mục tiêu

- Hiểu rõ khái niệm CI/CD (Continuous Integration/Continuous Deployment), vai trò của Jenkins, và hiểu **Unit Test là gì**, vì sao nó là một bước bắt buộc trong CI thực thụ chứ không chỉ đơn thuần là "build ra được là xong".
- Đưa được project Web API của nhóm lên GitHub ở chế độ **Private Repository**, và cấu hình để Jenkins (dù cài trên máy cá nhân) vẫn pull được code từ private repo đó.
- Có khả năng cài đặt Jenkins (trực tiếp trên Windows hoặc bằng Docker) và viết pipeline dưới dạng **Jenkinsfile** (Pipeline as Code) thay vì cấu hình rời rạc qua giao diện.
- Cấu hình được pipeline: checkout code → chạy Unit Test → build Docker image có gắn tag → push image lên một Registry (Docker Hub hoặc GitHub Container Registry).
- Hiểu vì sao chạy Jenkins trên máy cá nhân thì **không thể** có webhook tự động từ GitHub, phải bấm "Build Now" thủ công — nhưng vẫn mô phỏng được bước CD tự động bằng cách để Jenkins **chủ động gọi một HTTP webhook sang một máy triển khai (CD) riêng biệt** sau khi push image thành công.
- Biết cấu hình máy CD (máy thứ hai, đóng vai trò server triển khai) để lắng nghe webhook, tự pull image theo tag mới nhất và chạy migration để cập nhật database, dựng lên một Web API hoàn chỉnh có kết nối database.
- Chuẩn bị tốt cho phần thuyết trình và demo trên 2 máy riêng biệt: một máy đóng vai trò CI (Jenkins), một máy đóng vai trò CD (triển khai).

2. Nội dung cần nghiên cứu và chuẩn bị

Nội dung đi theo đúng luồng của một pipeline CI/CD thực tế được mô phỏng trên 2 máy cá nhân: Private repo → Jenkins build & test → Push image lên registry → Gọi webhook sang máy CD → Máy CD pull image, chạy migration và deploy.

2.1. Giới thiệu CI/CD và Jenkins

- **CI/CD:**
  - **Continuous Integration (CI):** Quá trình tự động hóa việc tích hợp code từ nhiều developer, bao gồm build, **chạy test tự động**, và kiểm tra chất lượng code trước khi cho phép code được đóng gói.
  - **Continuous Deployment/Delivery (CD):** Quá trình tự động hóa việc triển khai ứng dụng (image đã build) lên môi trường chạy thật sau khi CI thành công.
- **Jenkins:** Công cụ mã nguồn mở để tự động hóa các quy trình CI/CD, tích hợp với Git, Docker, và có thể gọi ra các hệ thống khác qua HTTP.
- **Vì sao demo bài này cần 2 máy riêng biệt (CI và CD):** Trong thực tế, máy chạy Jenkins (CI) và máy chạy ứng dụng thật (CD) thường là hai server khác nhau. Việc tách ra 2 máy khi demo giúp thấy rõ luồng: CI build xong không tự chạy ứng dụng tại chỗ, mà đóng gói thành image, đẩy lên registry rồi **gọi sang máy khác** để máy đó tự kéo về chạy — đúng như mô hình triển khai thật.

2.2. Chuẩn bị Project và Private Repository trên GitHub

- Đưa source code Web API của nhóm lên GitHub, thiết lập repository ở chế độ **Private**.
- **Vì sao để Private:** Mô phỏng đúng thực tế — code sản phẩm của công ty gần như luôn là private, Jenkins/CI muốn pull được code phải được cấp quyền truy cập rõ ràng (không thể clone công khai như repo public).

- Repo cần có sẵn: source code Web API, một test project (xem mục 2.5), Dockerfile (kế thừa từ Buổi 6), và file `Jenkinsfile` ở thư mục gốc.

2.3. Cài đặt Jenkins

- **Lựa chọn 1 — Cài trực tiếp trên Windows:** Tải Jenkins for Windows (bản `.msi`), cài đặt như một Windows service, truy cập giao diện qua `http://localhost:8080`.
- **Lựa chọn 2 — Chạy Jenkins bằng Docker:** `docker run -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts`. Lưu ý nếu muốn Jenkins tự build Docker image bên trong container của chính nó, cần mount thêm `docker.sock` (hoặc cài Docker CLI vào image Jenkins tùy chỉnh) để Jenkins gọi được lệnh `docker build`/`docker push` ra Docker Engine của máy host.
- **Cài plugin cần thiết:** Git, Pipeline, Docker Pipeline, Credentials Binding.
- **Không bắt buộc chọn một trong hai theo hệ điều hành cố định** — nhóm có thể chọn cách nào thuận tiện hơn, miễn giải thích được sự khác biệt khi thuyết trình.

2.4. Cấu hình Jenkins truy cập Private Repository

- **Deploy Key (SSH) — các bước cụ thể cần làm:**
  1. Trên máy Jenkins, tạo cặp SSH key riêng cho việc này (không dùng chung key cá nhân): `ssh-keygen -t ed25519 -C "jenkins-deploy-key" -f jenkins_deploy_key` (không đặt passphrase để Jenkins tự động dùng được).
  2. Vào repository trên GitHub → **Settings** → **Deploy keys** → **Add deploy key**, dán nội dung file public key (`jenkins_deploy_key.pub`), đặt tên gợi nhớ, **để nguyên "Allow write access" ở trạng thái tắt** vì Jenkins chỉ cần quyền đọc (clone) để build, không cần ghi ngược lại repo.
  3. Trên Jenkins, vào **Manage Jenkins** → **Credentials**, thêm một credential kiểu "SSH Username with private key", dán nội dung file private key (`jenkins_deploy_key`) vào.
  4. Trong cấu hình Job/Jenkinsfile, dùng URL dạng SSH của repo (`git@github.com:<user>/<repo>.git`) và chọn credential SSH vừa tạo — mỗi Deploy Key chỉ gắn được với **một** repository, phù hợp khi Jenkins chỉ cần build một project cố định.
- **Hoặc Personal Access Token (PAT — thay thế đơn giản hơn, phù hợp khi cần Jenkins truy cập nhiều repo hoặc cả push package):** Tạo PAT trên GitHub (**Settings** → **Developer settings** → **Personal access tokens**), chỉ cấp đúng quyền cần thiết (`repo` để clone private repo; nếu dùng chung PAT này để push image lên `ghcr.io` thì cấp thêm `write:packages` và `read:packages`), lưu vào Jenkins Credentials dạng "Username with password" (username là tài khoản GitHub, password là PAT) để Jenkins clone qua HTTPS.
- Trong Jenkinsfile, khai báo credentials này ở phần `checkout` để pipeline có quyền pull code private về.

2.5. Unit Test là gì và vì sao phải có trong pipeline

- **Unit Test (kiểm thử đơn vị):** Là các đoạn code tự động kiểm tra một đơn vị nhỏ nhất của ứng dụng (thường là một hàm/method) có hoạt động đúng như kỳ vọng hay không, chạy độc lập (không phụ thuộc database, network thật), chạy nhanh, và có thể chạy lại bất cứ lúc nào.
- **Vì sao cần trong CI:** Nếu pipeline chỉ "build ra được là xong" thì không khác gì build thủ công trên máy cá nhân — mất đi ý nghĩa của CI. Việc bắt buộc chạy Unit Test trong pipeline mô phỏng đúng vai trò thật của CI: **chỉ cho phép đóng gói (build image) khi code đã vượt qua các bài kiểm thử tự động**, nếu có test fail thì pipeline phải dừng lại ngay, không build/push image tiếp.

- **Yêu cầu cụ thể:** Project Web API của nhóm phải có một test project riêng (ví dụ xUnit/NUnit/MSTest cho .NET, Jest/Mocha cho Node.js, PyTest cho Python) với ít nhất vài test case kiểm tra logic nghiệp vụ cơ bản (không cần nhiều, quan trọng là chứng minh được luồng "test fail thì pipeline fail" hoạt động đúng).
- **Trong Jenkinsfile:** Có một `stage('Test')` riêng biệt chạy lệnh test (ví dụ `dotnet test`), đặt trước `stage('Build Image')`.

2.6. Jenkinsfile — Pipeline as Code

- **Jenkinsfile là gì:** File định nghĩa toàn bộ pipeline bằng code (Groovy DSL), được lưu ngay trong repository cùng với source code, thay vì cấu hình rời rạc qua giao diện Jenkins — giúp pipeline được version-control, review được như code bình thường.
- **Cấu trúc Declarative Pipeline cơ bản cần có trong bài này:**

```
pipeline {
    agent any
    stages {
        stage('Checkout') { ... }   // Pull code từ private repo
        stage('Test') { ... }       // Chạy unit test, fail thì dừng pipeline
        stage('Build Image') { ... }// docker build, gắn tag
        stage('Push Image') { ... } // docker login + docker push lên
registry
        stage('Trigger Deploy') { ... } // Gọi HTTP webhook sang máy CD
    }
}
```

- **Không có trigger tự động từ GitHub:** Vì Jenkins chạy trên máy cá nhân, không có domain/IP public để GitHub gọi webhook vào, nên pipeline **không** tự chạy khi có commit mới — sinh viên phải bấm "Build Now" thủ công trên giao diện Jenkins. Điều này không ảnh hưởng đến bước checkout code private, vì chiều gọi ngược lại (Jenkins chủ động ra ngoài gọi GitHub để clone) vẫn hoạt động bình thường, chỉ có chiều GitHub gọi vào Jenkins là không thực hiện được.

2.7. Build và Push Docker Image lên Registry

- **Build có gắn tag rõ ràng:** `docker build -t <registry>/<tên>/<app>:<tag> .`, trong đó `<tag>` nên dùng biến có sẵn của Jenkins (`${BUILD_NUMBER}`) hoặc short commit SHA để mỗi lần build ra một tag khác nhau, dễ truy vết phiên bản nào đang chạy production — tránh chỉ dùng mãi tag `latest`.
- **Chọn Registry để push image (một trong các lựa chọn sau):**
  - **Docker Hub:** Registry mặc định, phổ biến nhất, miễn phí cho repository public. Cần tạo tài khoản, tạo Access Token, `docker login` bằng token đó rồi `docker push`.
  - **GitHub Container Registry (`ghcr.io`):** Registry riêng của GitHub (nằm trong GitHub Packages), thuận tiện vì dùng chung Personal Access Token đã tạo ở mục 2.4, image gắn liền trực tiếp với repository trên GitHub. Đăng nhập bằng `docker login ghcr.io -u <username> -p <PAT>` (PAT cần quyền `write:packages`/`read:packages` như đã lưu ý ở mục 2.4).
    - `ghcr.io` **hoàn toàn hỗ trợ image private**, không bắt buộc phải để repo/image public — mặc định image mới push lên thường ở trạng thái private, có thể chỉnh trong phần Package settings trên GitHub nếu muốn public. Nếu nhóm không có nhu cầu image

private, dùng luôn ở chế độ public cũng được, dễ pull hơn từ máy CD (không cần `docker login` trên máy CD).
    - Nhóm có thể chọn một trong hai; nếu chọn Docker Hub thì cần cấu hình thêm Credentials cho Docker Hub trong Jenkins tương tự như đã làm với GitHub.

2.8. Trigger CD sang máy triển khai qua HTTP Webhook

- **Kịch bản 2 máy:** Máy A chạy Jenkins (CI), Máy B là máy triển khai thực tế (CD) — hai máy này có thể là 2 máy vật lý/VM khác nhau trong cùng mạng LAN khi demo.
- **Trên máy CD (Máy B), cấu hình sẵn từ trước, không liên quan gì đến Jenkins:**
    - Một service nhỏ lắng nghe HTTP (ví dụ một endpoint `POST /deploy`) — có thể viết bằng một script Node.js/Python đơn giản, hoặc dùng công cụ webhook có sẵn (`webhook` CLI, hoặc một job Jenkins agent riêng ở chế độ chờ).
    - Biến môi trường (connection string database, secret,...) đã được cấu hình sẵn trên máy này từ trước — vì vai trò của máy CD chỉ là "kéo image mới nhất về và chạy", không build/không biết gì về source code.
    - Khi endpoint nhận được request: chạy `docker pull <image>:<tag>` theo tag được gửi kèm trong request, sau đó `docker compose up -d` (hoặc `docker stop` + `docker run` lại container với image mới) để thay thế container cũ bằng phiên bản mới.
- **Trên Jenkinsfile (Máy A), `stage('Trigger Deploy')`:** Sau khi push image thành công, gọi một HTTP request (ví dụ bằng `curl -X POST http://<IP_máy_CD>:<port>/deploy -d '{"tag":"<tag_vừa_build>"}'`) để báo cho máy CD biết có phiên bản mới cần triển khai.
- **Ý nghĩa:** Đây chính là điểm mấu chốt mô phỏng CD trong điều kiện không có server thật/domain thật — CI chủ động "gõ cửa" CD thay vì chờ CD tự động phát hiện, nhưng vẫn giữ đúng nguyên tắc: build một lần, deploy image đã được kiểm thử, không build lại trên máy CD.

2.9. Image Web API hoàn chỉnh có Database và chạy Migration khi Deploy

- **Image không phải ví dụ đơn giản:** Image được build và triển khai trong bài này phải là một Web API hoàn chỉnh, có kết nối đến database thật (kế thừa từ các buổi Docker Compose trước), không phải một API "Hello World".
- **Chạy Migration khi deploy phiên bản mới:** Khi máy CD triển khai container mới, ứng dụng phải tự động (hoặc qua một bước riêng trong script deploy) chạy migration để cập nhật schema database theo đúng phiên bản code mới nhất — ví dụ gọi `dbContext.Database.Migrate()` lúc ứng dụng khởi động (với .NET/EF Core), hoặc chạy lệnh migrate riêng (Prisma, Sequelize, Alembic...) trước khi khởi động lại service.
- **Vì sao quan trọng:** Đảm bảo mỗi lần release version mới, database luôn đồng bộ với code đang chạy, tránh lỗi runtime do thiếu cột/bảng — đây là một phần không thể thiếu của một quy trình CI/CD thực tế cho ứng dụng có database.

3. Chuẩn bị thuyết trình và demo

- **Slide:** Giới thiệu CI/CD, Jenkins, Unit Test, Jenkinsfile, Registry, và toàn bộ luồng CI (Máy A) → CD (Máy B) theo đúng lộ trình ở mục 2.
- **Chuẩn bị bắt buộc:** Có sẵn một project Web API **hoàn chỉnh, chạy được thật** (không phải mã nguồn rời rạc/mẫu) làm mã nguồn demo, đủ để chứng minh toàn bộ luồng CI/CD tạo ra sản phẩm cuối cùng.
- **Demo — bắt buộc thực hiện trên 2 máy riêng biệt, đóng vai trò CI và CD:**

1. Đưa project Web API (có sẵn test project và Dockerfile) lên một GitHub repository ở chế độ Private.
2. Cài Jenkins trên Máy A (Windows trực tiếp hoặc Docker), cấu hình Deploy Key/PAT để Jenkins clone được private repo.
3. Viết `Jenkinsfile` với đầy đủ các stage: Checkout → Test → Build Image (có tag) → Push Image lên registry (Docker Hub hoặc `ghcr.io`) → Trigger Deploy (gọi HTTP sang Máy B).
4. **Bắt buộc:** Chứng minh stage Test có tác dụng thật — sửa một test case cho fail, chạy pipeline và cho thấy pipeline dừng lại ở stage Test, không build/push image. Sau đó sửa lại đúng và chạy lại thành công.
5. Trên Máy B (máy CD), cấu hình sẵn webhook listener và biến môi trường, chứng minh: khi Jenkins ở Máy A chạy xong và gọi webhook, Máy B tự động `docker pull` image theo đúng tag vừa build và khởi động lại container.
6. Kiểm tra ứng dụng Web API đang chạy trên Máy B hoạt động đúng, gọi thử API thật, và chỉ ra rằng migration đã được áp dụng lên database (ví dụ: một bảng/cột mới xuất hiện đúng theo migration mới nhất trong code).
7. Vì Jenkins chạy trên máy cá nhân nên **không có bước** GitHub tự động gọi webhook vào Jenkins — nhấn mạnh rõ điều này khi demo, chạy build bằng cách bấm "Build Now" thủ công.
8. **Bắt buộc — minh họa "thêm code mới thì ra sản phẩm mới như thế nào":** Ngay trong buổi thuyết trình, thêm một thay đổi nhỏ vào source code (ví dụ thêm/sửa một endpoint), commit và push lên GitHub, sau đó bấm "Build Now" và đi qua toàn bộ pipeline (Test → Build → Push → Trigger Deploy) trước mặt người xem, kết thúc bằng việc gọi thử API trên Máy B để thấy thay đổi vừa thêm đã có hiệu lực trên bản deploy mới — đây là phần chứng minh quan trọng nhất của cả buổi, không chỉ demo lại các bước đã chuẩn bị sẵn.

- **Thời gian:** Phân bổ thời gian hợp lý cho lý thuyết và demo, ưu tiên thời lượng cho bước 4 (chứng minh Test thật sự chặn pipeline) và bước 5 (luồng CI gọi sang CD).

4. Phân công công việc và ước lượng thời gian (Gợi ý)

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Nghiên cứu về CI/CD, Jenkins, Unit Test	TV1	3
Nghiên cứu Private Repo, Deploy Key/PAT	TV2	2
Nghiên cứu Jenkinsfile (Pipeline as Code) và cú pháp Declarative Pipeline	TV1	3
Nghiên cứu Registry (Docker Hub / GitHub Container Registry)	TV2	2
Chuẩn bị nội dung slide thuyết trình	Nhóm trưởng	2
Viết test project (unit test) cho Web API demo	TV3	3
Cài đặt Jenkins (Máy A) và cấu hình truy cập private repo	TV1	3
Viết Jenkinsfile đầy đủ các stage (Checkout, Test, Build, Push, Trigger Deploy)	TV2, TV3	6
Cấu hình Máy B (webhook listener, env, pull image, chạy migration)	TV3	5

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Tổng hợp, kiểm tra và chạy thử demo trên 2 máy	Cả nhóm	3
Luyện tập thuyết trình và demo	Cả nhóm	1
Tổng cộng		**33**

5. Tài liệu tham khảo

- **Tài liệu chính thức:** Jenkins Documentation (jenkins.io/doc/), cú pháp Jenkinsfile/Declarative Pipeline, Docker Hub Documentation, GitHub Container Registry Documentation (docs.github.com/packages).
- **Bài viết:** Các bài hướng dẫn cài đặt Jenkins (Windows/Docker), viết Unit Test cơ bản theo ngôn ngữ backend của nhóm, cấu hình Deploy Key/Personal Access Token, gọi webhook bằng `curl` trong pipeline.
- **Video:** Các video hướng dẫn CI/CD với Jenkins, viết Jenkinsfile, và cấu hình deploy image qua webhook giữa 2 server.

6. Ghi chú quan trọng

- **Bảo mật:** Luôn bảo mật private key, Personal Access Token và các thông tin xác thực khác; lưu trong Jenkins Credentials, không hard-code trong Jenkinsfile hay commit lên GitHub.
- **Không dùng mãi tag `latest`:** Luôn gắn tag rõ ràng (build number/commit SHA) cho mỗi image để biết chính xác phiên bản nào đang chạy trên máy CD, dễ rollback khi cần.
- **Unit Test phải có ý nghĩa thật:** Đừng viết test chỉ để "cho có" — phải chứng minh được khi test fail thì pipeline thực sự dừng lại, đây là phần giám khảo sẽ kiểm tra kỹ.
- **Backup:** Backup cấu hình Jenkins (hoặc ít nhất lưu lại Jenkinsfile trong repo) để không mất công cấu hình lại từ đầu.
- **Monitoring:** Theo dõi log của Jenkins (console output của từng build) và log ứng dụng trên máy CD để phát hiện, xử lý lỗi kịp thời khi pipeline hoặc deploy thất bại.
- **Migration an toàn:** Cẩn trọng khi để ứng dụng tự động chạy migration lúc khởi động — trong thực tế cần đảm bảo migration không phá vỡ dữ liệu hiện có; ở mức bài học, giải thích được cơ chế là đủ.

Chúc các bạn học tập hiệu quả và thành công!